Guía Completa de Programación en R

Desde los Fundamentos Básicos hasta el Ecosistema Tidyverse y Visualización Avanzada

> 📌 **Enlaces Oficiales de Descarga y Recurso en Video**
> 1. R Project (Lenguaje R): https://www.r-project.org/
> 2. RStudio Desktop (IDE): https://posit.co/download/rstudio-desktop/
> 3. Videotutorial de Referencia (Aprende R en 90 minutos): https://youtu.be/WCcHcWsMN1o?si=TS52w_BBZertmraF

1. Introducción e Instalación del Entorno de Trabajo

R es un lenguaje de programación y entorno de software libre especializado en análisis estadístico, minería de datos, gráficos y ciencia de datos. Para trabajar de manera eficiente y profesional, se requiere instalar dos componentes fundamentales:

• **R (CRAN):** El motor principal del lenguaje que ejecuta todo el código y realiza los cálculos numéricos y estadísticos.

• **RStudio Desktop:** El entorno de desarrollo integrado (IDE) que facilita la escritura de scripts, la visualización de gráficos, la gestión de paquetes y la inspección de variables.

Pasos para la Instalación:

1. Ve a r-project.org, selecciona un espejo CRAN (por ejemplo, el de tu país o la versión global) y descarga el instalador correspondiente a tu sistema operativo (Windows, macOS o Linux). Ejecuta el instalador siguiendo los pasos predeterminados.

2. Una vez instalado R, dirígete a posit.co (antes RStudio), descarga RStudio Desktop (versión gratuita) e instálalo.

3. Abre RStudio. El IDE detectará automáticamente la versión de R instalada en tu sistema.

Estructura de la Interfaz de RStudio:

• **Editor de Scripts (Arriba a la izquierda):** Ventana donde se escriben y guardan las líneas de código reutilizables (extensión .R).

• **Consola (Abajo a la izquierda / Arriba a la derecha según personalización):** Espacio interactivo donde el código se ejecuta en tiempo real y muestra los resultados inmediatos.

• **Entorno de Trabajo (Environment - Arriba a la derecha):** Muestra las variables, listas, funciones y data frames almacenados en memoria.

• **Panel de Salida (Abajo a la derecha):** Pestañas multifunción para explorar archivos locales, visualizar gráficos (Plots), gestionar librerías instaladas (Packages) y consultar documentación oficial (Help).

> **Consejo de Personalización**
> Puedes personalizar el tema visual y la tipografía en Tools > Global Options > Appearance. Para reorganizar los paneles (por ejemplo, situar la Consola arriba a la derecha), dirígete a Tools > Global Options > Pane Layout.

2. Primeros Pasos: Uso de R como Calculadora y Variables

R permite realizar operaciones matemáticas básicas y complejas de forma directa en la consola o desde un script.

```
# Operaciones matemáticas elementales
5 + 3          # Suma -> 8
10 - 4         # Resta -> 6
4 * 2          # Multiplicación -> 8
20 / 4         # División -> 5
2 ** 3         # Exponenciación (o 2^3) -> 8

# Funciones matemáticas integradas
abs(-15)       # Valor absoluto -> 15
log(10)        # Logaritmo natural
sqrt(49)       # Raíz cuadrada -> 7
round(3.14159, 2) # Redondeo a 2 decimales -> 3.14
ceiling(2.3)   # Entero mayor más cercano -> 3
floor(2.8)     # Entero menor más cercano -> 2
trunc(2.9)     # Trunca la parte decimal -> 2
```

Asignación de Variables:

Para almacenar resultados en memoria y reutilizarlos, se asignan valores a objetos mediante el operador <- (combinación de teclas Alt + - en Windows o Option + - en Mac). También se puede usar =, pero <- es el estándar en R.

```
# Ejemplo: Cálculo del Índice de Masa Corporal (IMC)
peso <- 83
estatura <- 1.85
imc <- peso / (estatura ** 2)
print(imc) # Muestra ~24.25 en consola
```

3. Tipos de Datos Fundamentales

R maneja varios tipos de datos atómicos básicos:

1. Numéricos (Numeric): Números reales con o sin decimales (ej. 20, -15, 3.1416).

2. Caracteres / Strings (Character): Cadenas de texto encerradas entre comillas simples o dobles (ej. 'Bioinformática', "RStudio").

3. Lógicos (Logical): Valores booleanos TRUE (Verdadero) o FALSE (Falso). Internalmente, TRUE equivale a 1 y FALSE a 0.

Manipulación de Strings (Cadenas de Texto):

```
texto <- 'Programación en R'
nchar(texto)                    # Número de caracteres -> 17
tolower(texto)                  # Convertir a minúsculas
toupper(texto)                  # Convertir a mayúsculas
substr(texto, start=1, stop=12) # Extraer subcadena -> 'Programación'
sub('R', 'RStudio', texto)      # Reemplaza la primera ocurrencia
gsub('a', 'X', texto)           # Reemplaza todas las ocurrencias
paste('Aprender', 'R', sep=' ') # Concatena cadenas con separador
```

Operadores Lógicos y Comparaciones:

```
calificacion <- 8.5
calificacion >= 6               # TRUE (Aprobado)
calificacion == 10              # FALSE (No es 10 perfecto)
calificacion != 5               # TRUE (Es diferente de 5)

# Operadores lógicos combinados: AND (&) y OR (|)
(calificacion > 6) & (calificacion < 9) # TRUE
(calificacion == 10) | (calificacion > 8) # TRUE
```

4. Estructuras de Datos Avanzadas

4.1 Vectores (Vectores Atómicos)

Un vector es la estructura de datos más básica en R y almacena múltiples elementos del mismo tipo. Se crea con la función c() (combinar). R es un lenguaje vectorizado, por lo que las operaciones se aplican elemento a elemento de forma automática.

```
# Crear vectores
vistas <- c(8321, 5400, 6100, 2974, 7200)
nombres <- c('Video 1', 'Video 2', 'Video 3', 'Video 4', 'Video 5')
```

```
# Funciones sobre vectores
length(vistas)                # Número de elementos -> 5
max(vistas)                   # Valor máximo -> 8321
min(vistas)                   # Valor mínimo -> 2974
mean(vistas)                  # Promedio -> 6000
sum(vistas)                   # Suma total
sort(vistas, decreasing=TRUE)  # Ordenar de mayor a menor

# Indexación y filtrado de vectores
vistas[1]                     # Primer elemento
vistas[c(1, 3)]               # Elementos en posiciones 1 y 3
vistas[-2]                    # Todos excepto la posición 2
vistas[vistas > 6000]         # Filtrado condicional -> valores mayores a 6000
which(vistas > 6000)          # Índices donde la condición es TRUE
```

4.2 Listas

A diferencia de los vectores, las listas son estructuras heterogéneas que pueden almacenar objetos de diferentes tipos y tamaños (números, strings, vectores, otros objetos e incluso data frames).

```
# Definir una lista con nombres
mi_lista <- list(
  usuario = 'Luis',
  edad = 35,
  puntuaciones = c(90, 95, 88),
  activo = TRUE
)

# Acceder a elementos de una lista
mi_lista$usuario              # Acceso por nombre mediante el operador $
mi_lista[[1]]                 # Acceso por posición usando doble corchete [[ ]]
mi_lista$puntuaciones[2]      # Accede al segundo valor del vector interno -> 95
```

4.3 Data Frames

El Data Frame es la estructura fundamental para el análisis de datos bidimensionales (tablas con filas y columnas). Cada columna es un vector de un tipo específico de datos, pero diferentes columnas pueden tener diferentes tipos.

```
# Crear un Data Frame desde cero
series <- data.frame(
  nombre = c('Breaking Bad', 'The Office', 'Invincible'),
  imdb = c(9.5, 8.9, 8.7),
  episodios = c(62, 201, 16),
  concluida = c(TRUE, TRUE, FALSE)
)

# Exploración básica del Data Frame
head(series, 2)               # Muestra las primeras 2 filas
str(series)                   # Estructura del Data Frame
```

```r
summary(series)                # Resumen estadístico por columna
nrow(series)                   # Número de filas
ncol(series)                   # Número de columnas

# Selección e indexación
series$imdb                    # Extrae la columna imdb como vector
series[1, ]                    # Primera fila completa
series[, 'nombre']             # Columna 'nombre'
series[series$imdb >= 9.0, ]   # Filtrar filas donde imdb >= 9.0

# Guardar y leer archivos CSV
write.csv(series, 'series.csv', row.names = FALSE)
datos_guardados <- read.csv('series.csv')
```

5. Control de Flujo y Funciones Personalizadas

5.1 Estructuras Condicionales (IF / ELSE)

Permiten ejecutar bloques de código específicos en función de si se cumple una condición lógica.

```r
elo <- 950

if (elo >= 750) {
  rango <- 'Gran Maestro'
} else if (elo >= 500) {
  rango <- 'Maestro'
} else if (elo >= 250) {
  rango <- 'Profesional'
} else {
  rango <- 'Novato'
}
print(rango)

# Versión vectorizada: ifelse()
numeros <- c(1, 2, 3, 4, 5, 6)
paridad <- ifelse(numeros %% 2 == 0, 'Par', 'Impar')
```

5.2 Creación de Funciones Personalizadas

Las funciones permiten encapsular lógica reutilizable, reduciendo la duplicación de código y mejorando la legibilidad.

```r
# Ejemplo 1: Conversión de grados Celsius a Fahrenheit
celsius_a_fahrenheit <- function(celsius) {
  fahrenheit <- (celsius * 9/5) + 32
  return(fahrenheit)
}
celsius_a_fahrenheit(30) # Devuelve 86
```

```
# Ejemplo 2: Función con parámetro por defecto y validación condicional
actualizar_semaforo <- function(color_actual = 'verde') {
  color <- tolower(color_actual)
  if (color == 'verde') {
    return('amarillo')
  } else if (color == 'amarillo') {
    return('rojo')
  } else if (color == 'rojo') {
    return('verde')
  } else {
    stop('Color no válido. Ingresa verde, amarillo o rojo.')
  }
}
actualizar_semaforo('amarillo') # Devuelve 'rojo'
```

5.3 Bucle FOR y Bucle WHILE

```
# Bucle FOR sobre un rango numérico
for (i in 1:5) {
  print(paste('Iteración número:', i))
}

# Bucle WHILE condicional
contador <- 1
while (contador <= 3) {
  print(paste('Contador en:', contador))
  contador <- contador + 1
}
```

6. El Ecosistema Tidyverse

Tidyverse es una colección armoniosa de paquetes diseñados para la ciencia de datos en R. Comparte una filosofía común de diseño basada en estructuras de datos 'limpias' (tidy data).

```
# Instalación y carga del ecosistema Tidyverse completo
install.packages('tidyverse')
library(tidyverse)
```

6.1 El Operador Pipe (%>% o |>)

El operador Pipe (%>%) encadena operaciones enviando el resultado del lado izquierdo como primer argumento de la función del lado derecho. Esto elimina el anidamiento complejo de funciones y hace que el código se lea de izquierda a derecha de forma secuencial.

```
# Ejemplo sin Pipe (Anidado y difícil de leer):
resultado <- nrow(filter(read.csv('videogames.csv'), platform == 'PC'))
```

```
# Ejemplo equivalente CON Pipe (Claro, modular y secuencial):
resultado <- read.csv('videogames.csv') %>%
  filter(platform == 'PC') %>%
  nrow()
```

6.2 Manipulación de Datos con dplyr

dplyr proporciona una 'gramática para la manipulación de datos' basada en verbos clave:

• **filter():** Filtra filas basándose en condiciones lógicas.

• **select():** Selecciona columnas específicas por nombre.

• **mutate():** Crea nuevas columnas o modifica las existentes.

• **arrange():** Ordena filas según los valores de una o más columnas.

• **group_by() + summarize():** Agrupa y resume los datos calculando métricas agregadas (promedio, suma, conteo).

```
# Ejemplo práctico de pipeline completo con dplyr:
resumen_videojuegos <- read_csv('videogames.csv') %>%
  # Limpieza de espacios en blanco en la columna platform
  mutate(platform = str_trim(platform)) %>%
  # Filtrar solo juegos con metascore alto
  filter(meta_score >= 80) %>%
  # Agrupar por plataforma
  group_by(platform) %>%
  # Calcular métricas resumen
  summarize(
    total_juegos = n(),
    promedio_metascore = mean(meta_score, na.rm = TRUE)
  ) %>%
  # Ordenar de mayor a menor según el promedio del metascore
  arrange(desc(promedio_metascore))

print(resumen_videojuegos)
```

7. Visualización de Datos con R Base y ggplot2

7.1 Gráficos Básicos con R Base

R incluye funciones nativas de visualización para exploración rápida sin requerir librerías externas.

```
# Histograma de distribución
hist(series$imdb, breaks = 5, col = 'lightblue', main = 'Distribución de IMDB', xlab =
'Calificación')
```

```
# Gráfico de dispersión (Scatter plot)
plot(series$episodios, series$imdb, col = 'blue', pch = 19, main = 'Episodios vs
IMDB')

# Diagrama de caja y bigotes (Boxplot)
boxplot(meta_score ~ platform, data = datos_juegos, col = 'lightgreen', main =
'Metascore por Plataforma')
```

7.2 Visualización Avanzada con ggplot2

ggplot2 implementa la 'Gramática de Gráficos' (Grammar of Graphics). Los gráficos se construyen superponiendo capas conectadas mediante el operador +:

1. Datos (data): Define el dataset de entrada.

2. Mapeo Estético (aes): Asigna variables a los ejes x, y, color, forma o tamaño.

3. Capas Geométricas (geom_*): Especifica el tipo de representación visual (geom_histogram, geom_point, geom_boxplot, geom_bar).

4. Aspecto Visual: Define títulos, etiquetas de ejes y temas de diseño (theme_minimal, theme_bw).

```
# Ejemplo 1: Histograma moderno con ggplot2
ggplot(datos_juegos, aes(x = meta_score)) +
  geom_histogram(binwidth = 5, fill = '#2B6CB0', color = 'white', alpha = 0.8) +
  labs(title = 'Distribución del Meta Score en Videojuegos',
       x = 'Meta Score',
       y = 'Frecuencia') +
  theme_minimal()

# Ejemplo 2: Gráfico de dispersión estilizado
ggplot(datos_juegos, aes(x = meta_score, y = user_score, color = platform)) +
  geom_point(size = 2.5, alpha = 0.7) +
  labs(title = 'Puntuación Críticos vs Usuarios',
       x = 'Meta Score (Críticos)',
       y = 'User Score (Usuarios)') +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = 'bold'))

# Ejemplo 3: Boxplot por categorías
ggplot(datos_juegos, aes(x = platform, y = meta_score, fill = platform)) +
  geom_boxplot(alpha = 0.7) +
  labs(title = 'Distribución de Calificaciones por Plataforma',
       x = 'Plataforma',
       y = 'Meta Score') +
  theme_minimal() +
  theme(legend.position = 'none')
```

8. Buenas Prácticas de Programación en R

1. Nombres Claros y Consistentes: Usa snake_case (ej. mi_variable, calcular_promedio) para mantener consistencia.

2. Comentarios Estructurados: Utiliza el símbolo # para explicar el propósito de bloques de código complejos y la lógica detrás de tus análisis.

3. Indentación y Formato: Mantén la sangría (2 espacios por defecto) dentro de funciones, bucles e instrucciones condicionales.

4. Reproducibilidad: Siempre que trabajes con simulaciones o funciones aleatorias (sample, rnorm, runif), fija la semilla con set.seed(123) para garantizar que tus resultados sean reproducibles.

5. Código Vectorizado: Aprovecha las capacidades vectorizadas de R y las funciones de Tidyverse/dplyr antes de recurrir a bucles explicitos (for/while).

> 📌 **Conclusión y Próximos Pasos**
> ¡Felicidades por completar esta guía! Con estas bases sólidas estás listo para adentrarte en el mundo de la ciencia de datos, análisis bioinformático, modelado estadístico y aprendizaje automático utilizando R.